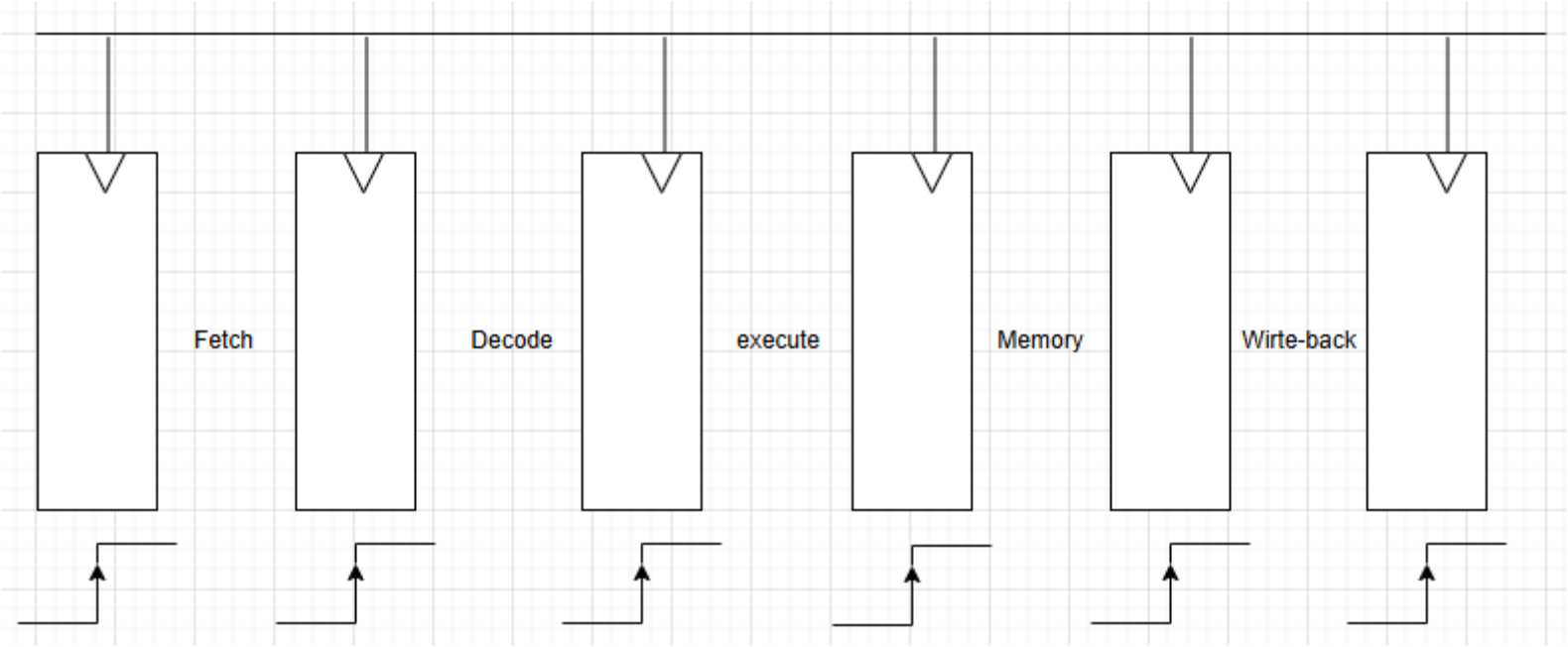
RISC-V RV32I

MCU 설계

개요



기존에 설계했던 RISC-V Single-Cycle에서 Register을 추가하여 총 5단계로 구성

(이전 프로젝트 : https://github.com/mskim991011/riscv-cpu-design.git)

WHY?

Single Cycle은 한 클럭 안에 [주소 해독 -> 데이터 전송 -> 결과 저장]을 모두 마침

- 임계 경로(Critical Path)의 문제: 모든 동작을 한 클럭에 하려면 전기 신호가 이동해야 하는 경로가 길다
- 결과: 경로가 길어지면 신호가 도착할 때까지 기다려야 하기 때문에 전체 시스템의 클럭 속도가 느려질 수 있음
- 멀티 사이클의 해결책: 과정을 2단계(Setup, Access)로 쪼개면 각 단계의 경로가 짧아지고 훨씬 더 높은 클럭 주파수에서도 시스템이 안정적으로 돌아감

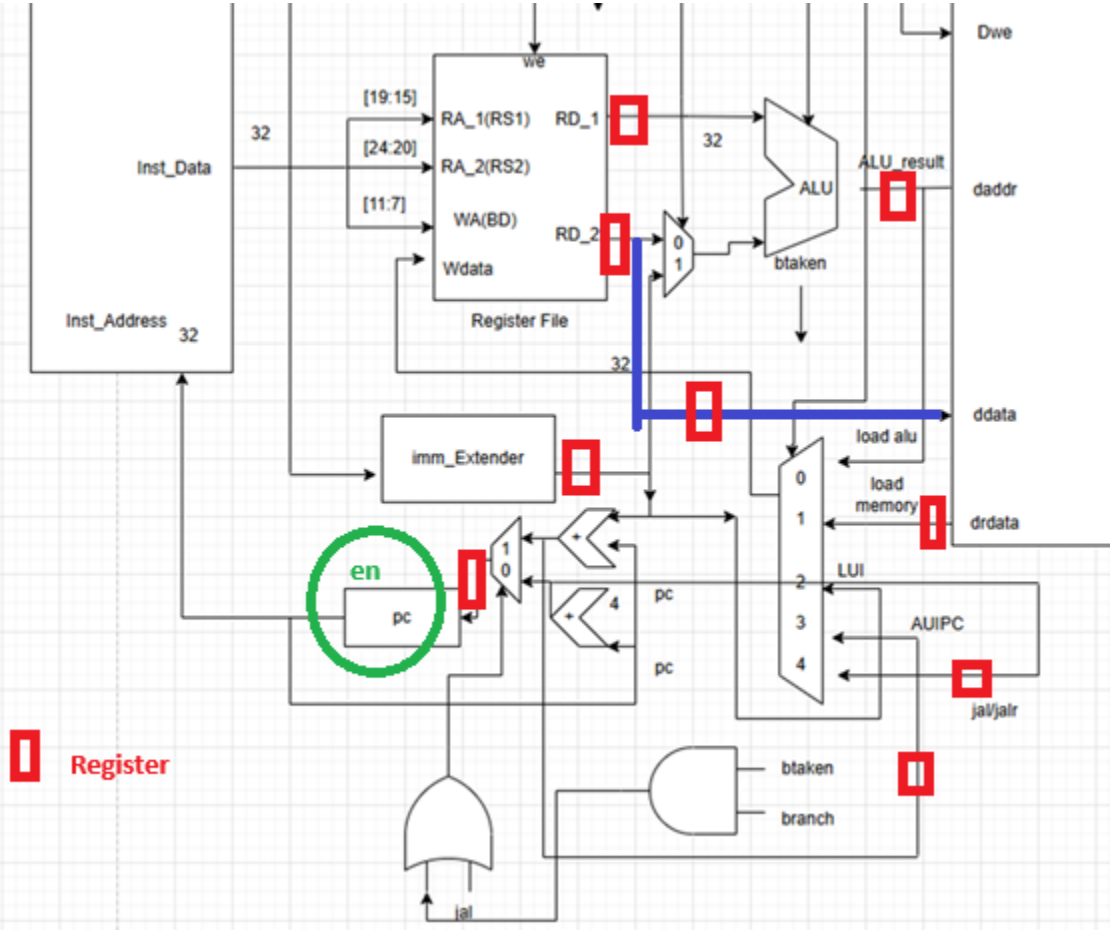
Multi-Cycle

1. **Fetch** : ROM으로 부터 명령어 받기
   ex) PC 값, pc_en
2. **Decode** : 명령어 분석
   ex) control unit, register file 입력, imm, frwe = 0
3. **Execute** : 동작 수행
   ex) ALU, next pc(비교기), branch 처리
4. **Memory** : data memory 접근
   ex) Addr, read/write, funct3 결정
5. **Write-back** : 데이터를 regiter file
   ex) alu source 선택, frwe = 1

Multi-Cycle

한 clk에 끝내는 single-cycle 과 다르게
여러 개의 clk로 나누어 실행

-> **중간에 값을 저장하고 결과를 넘겨주기 위해 Register 배치**



Multi-Cycle

```
module register_en (
    input                clk,
    input                reset,
    input                en,
    input  logic [31:0] data_in,
    output       [31:0] data_out
);
    logic [31:0] register;

    always_ff @(posedge clk, posedge reset) begin
        if (reset) begin
            register <= 0;
        end else begin
            if (en) register <= data_in;
        end
    end

    assign data_out = register;
endmodule
```

다음 상태로 넘어가기 위한 신호 선언

현재 실행 상태 보존
명령어가 진행중 일대는 pc_en =0 유지

```
register_en U_PC_REG (
    .clk(clk),
    .reset(reset),
    .en(pc_en),
    .data_in(pc_next),
    .data_out(program_counter)
);
```

기존 pc 레지스터 수정

Multi-Cycle

```
//execute
register U_EXE_REG_RS2 (
    .clk(clk),
    .reset(reset),
    .data_in(o_dec_rs2),  //from alu result
    .data_out(o_exe_rs2)  //to Data MEM_Wdata
);
register U_EXE_ALU_RESULT (
    .clk(clk),
    .reset(reset),
    .data_in(alu_result),
    .data_out(o_exe_alu_result)  //to daddr & mux in 0
);
```

레지스터 인스턴스하여 설계

```
//decode
register_file U_REG_FILE (
    .clk(clk),
    .reset(reset),
    .RA1(instr_data[19:15]),
    .RA2(instr_data[24:20]),
    .WA(instr_data[11:7]),
    .Wdata(rf_wb_data),
    .rf_we(rf_we),
    .RD1(i_dec_rs1),
    .RD2(i_dec_rs2)
);
imm_extender U_EXTENDER (
    .instr_data(instr_data),
    .imm_data  (imm_data)
);

register U_DEC_REG_RS1 (
    .clk(clk),
    .reset(reset),
    .data_in(i_dec_rs1),
    .data_out(o_dec_rs1)
);
register U_DEC_REG_RS2 (
    .clk(clk),
    .reset(reset),
    .data_in(i_dec_rs2),
    .data_out(o_dec_rs2)
);
register U_DEC_IMM_EXT (
    .clk(clk),
    .reset(reset),
    .data_in(imm_data),
    .data_out(o_dec_imm)
);
```

Multi-Cycle

Write-back을 위한 Register

```
//MEM to Writeback
register U_MEM_REG_DRDATA (
    .clk(clk),
    .reset(reset),
    .data_in(drdata),
    .data_out(o_mem_drdata) // mux in1
);
register U_EXE_LUI_DATA (
    .clk(clk),
    .reset(reset),
    .data_in(o_dec_imm),
    .data_out(o_exe_lui_data) //MUX in2
);
register U_EXE_PC_IMM (
    .clk(clk),
    .reset(reset),
    .data_in(pc_imm_out),
    .data_out(o_exe_pc_imm) //  MUX in3
);
register U_EXE_PC_4 (
    .clk(clk),
    .reset(reset),
    .data_in(pc_4_out),
    .data_out(o_exe_pc_4) // MUX in4
);
```

Multi-Cycle

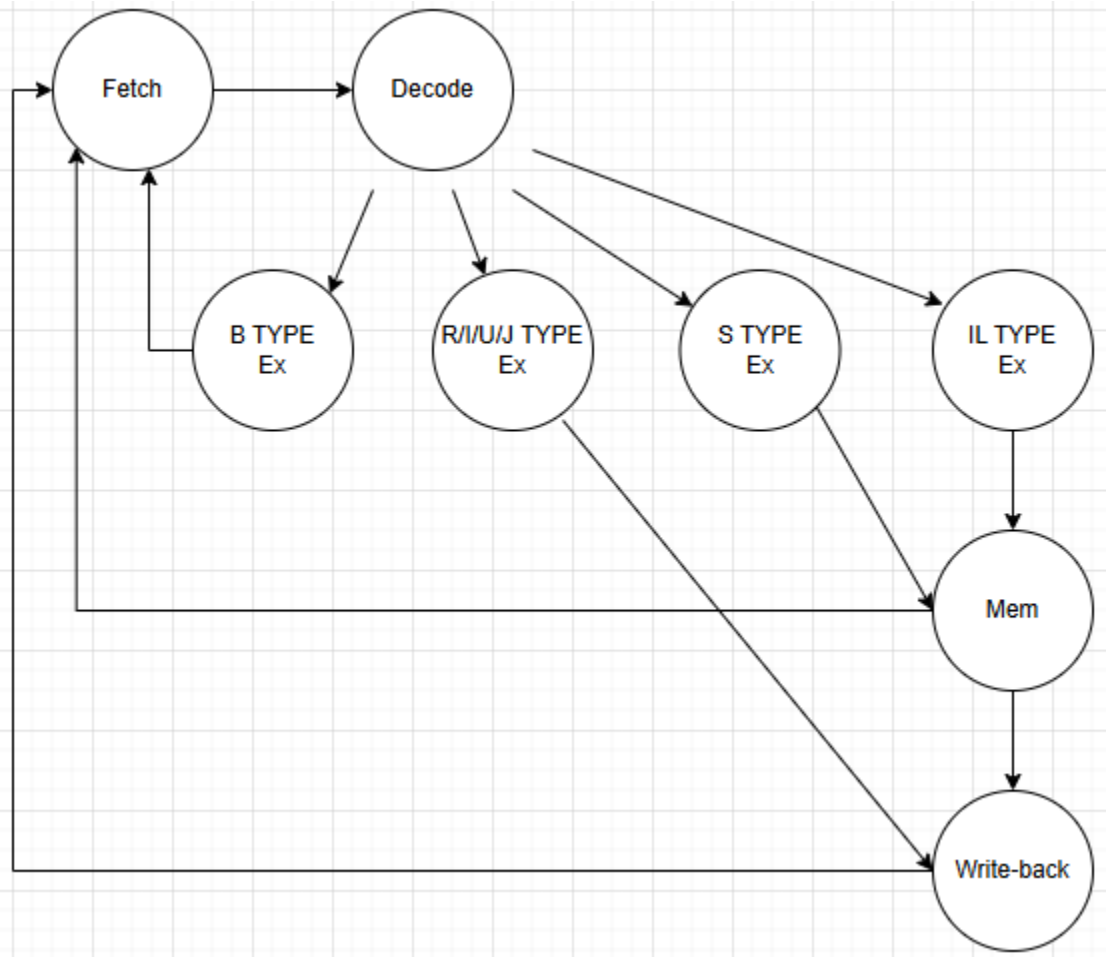
Control Unit FSM 구조

1. B
분기점(branch) 조건 맞는 비교하고 PC값 바꿈

2. R/I/J/U
계산하고 레지스터에 값을 쓰는 공통점
메모리 읽거나 쓰기 X

3. S
계산 후 데이터 쓰기
레지스터 파일에 값 쓰지 않으니 WB X

4. IL
계산 -> 읽기 -> 레지스터 저장 전부 수행



FSM

```
typedef enum logic [3:0] {
    FETCH,
    DECODE,
    EXE_RIUJ,
    EXE_L,
    EXE_S,
    EXE_B,
    MEM_S,
    MEM_L,
    WB
} state_e;
state_e c_state, n_state;
```

```
always_ff @(posedge clk, posedge reset) begin
    if (reset) begin
        c_state <= FETCH;
    end else begin
        c_state <= n_state;
    end
end
```

STATE 9개를 정의하여 순차 회로로 구성

FSM

```
always_comb begin
    n_state = c_state;
    fsm_rfwe   = 1'b0;
    fsm_dwe    = 1'b0;
    pc_en   = 1'b0;
    case (c_state)
        FETCH: begin
            n_state = DECODE;
        end
        DECODE: begin
            case (opcode)
                `B_TYPE:  n_state = EXE_B;
                `S_TYPE:  n_state = EXE_S;
                `IL_TYPE: n_state = EXE_L;
                default:  n_state = EXE_RIUJ;
            endcase
        end
```

기존 순차 회로와 겹치지 않게
rfwe 와 dwe 새로운 신호 정의

FETCH -> DECODE 후
TYPE 별로 다음 상태 지정

FSM

다음 상태가 FETCH인 경우
새 명령어를 위해 pc_en = 1

MEM -> FETCH 인 경우
계산을 끝내고 메모리 앞에 도착
dwe = 1

WB인 경우
계산과 메모리 읽기 끝내고 레지스터 앞
rfwe = 1

```
EXE_RIUJ: begin
    n_state = WB;
end
EXE_L: begin
    n_state = MEM_L;
end
EXE_S: begin
    n_state = MEM_S;
end
EXE_B: begin
    n_state = FETCH;
    pc_en   = 1'b1;
end
MEM_S: begin
    n_state = FETCH;
    fsm_dwe = 1'b1;
    pc_en = 1'b1;
end
MEM_L: begin
    n_state = WB;
end
WB: begin
    n_state = FETCH;
    fsm_rfwe   = 1'b1;
    pc_en   = 1'b1;
end
```

FSM

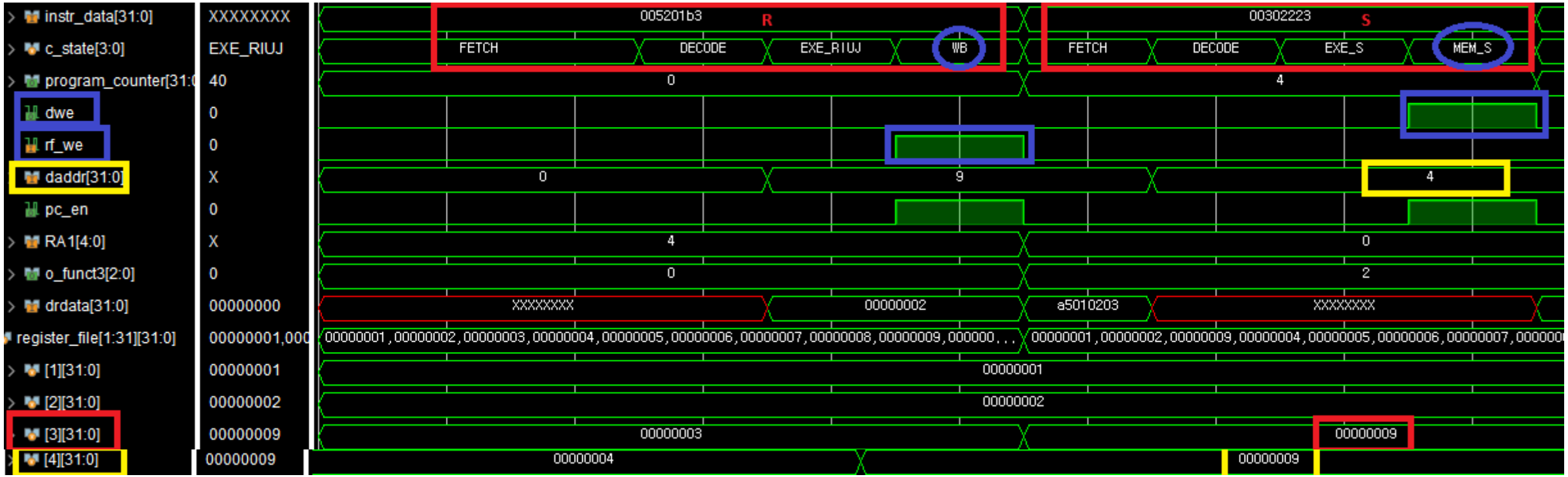
```
always_comb begin
    op_rfwe = 1'b0;
    alu_src = 1'b0;
    alu_control = 4'b0000;
    rfwd_src = 3'b000;
    o_funct3 = 3'b000;
    op_dwe = 1'b0;
    branch = 1'b0;
    jal = 1'b0;
    jalr = 1'b0;
    case (opcode)
```

기존 opcode로 나누었던 순차 회로

```
assign rf_we = fsm_rfwe & op_rfwe;
assign dwe = fsm_dwe & op_dwe;
```

출력을 위해 AND Gate 구성

Multi-Cycle Simulation

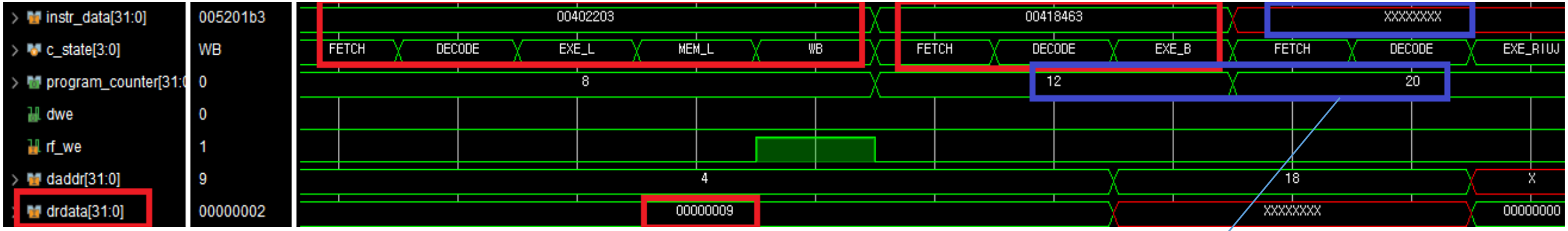



```
rom[0] = 32'h005201B3; // add x3, x4, x5
rom[1] = 32'h00302223; // sw x3, 4(x0)
```

R - x3 = x4 + 5

S – x3(9) 의 값 4번지에 저장

Multi-Cycle Simulation



점프 동작(addi 건너뜀)

```
rom[2] = 32'h00402203; // lw x4, 4(x0)
rom[3] = 32'h00418463; // beq x3, x4, 8 (PC = PC + 8)
rom[4] = 32'h06300293; // addi x5, x0, 99
```

IL – 메모리 4번지에서 값(9) 가져와 덮어쓰기(drdata = 9)
B – x3(9) = x4(9) 이므로 pc +8 동작

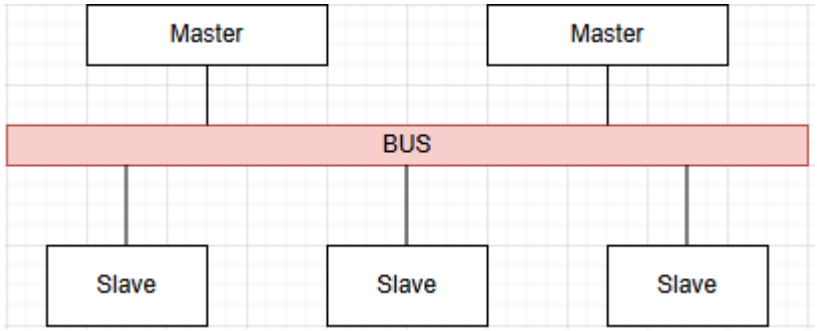
Bus

현재 구조의 문제점

- 새로운 장치(LED, 스위치, FND 등)을 추가하게 되면 입출력 포트 혹은 내부 MUX를 새로 설계해야 되기 때문에 매우 복잡

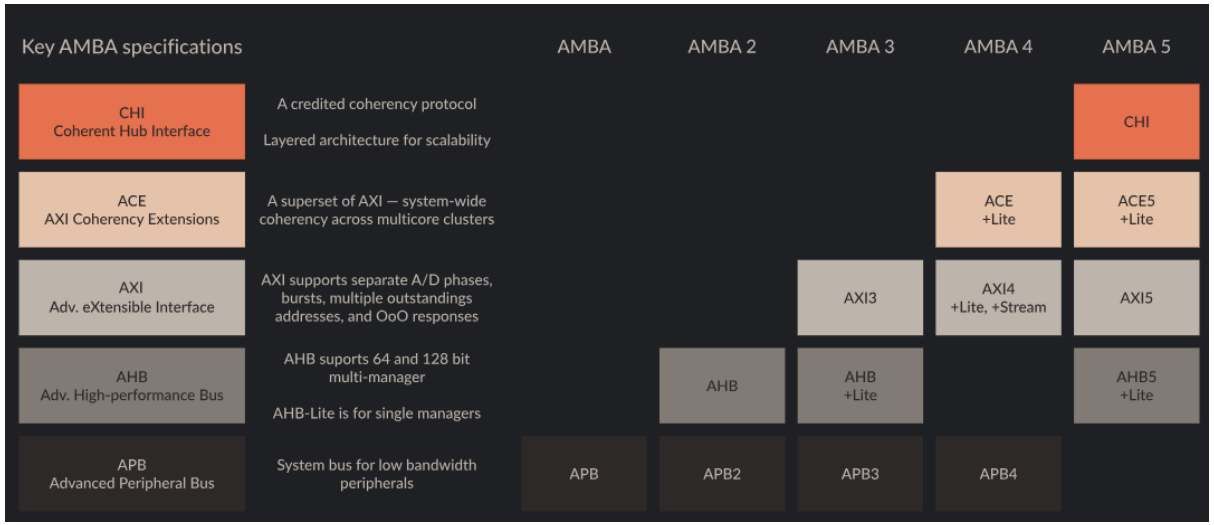
- CLK에 맞춘 CPU와 사람 속도인 외부 IO 장치 사이에서 생기는 큰 속도 차이

이를 해결하기 위해 산업 표준 버스 프로토콜을 설계하여 확장성 높은 SoC 아키텍처 완성



Bus

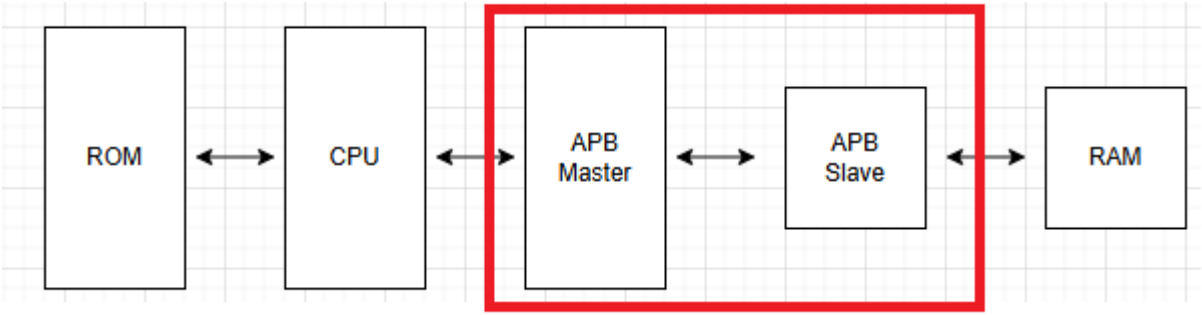
- Advanced Microcontroller Bus Architecture (AMBA)



출처 :
HTTPS://DEVELOPER.ARM.COM/DOCUMENTATION/102202/0300/WHAT-IS-AMBA--AND-WHY-USE-IT-

ARM사에서 제공하는 BUS의 종류는 위와 같고 본 프로젝트에서는
APB BUS를 설계

APB Bus



CPU로부터 Master가 제어 신호를 받고 처리한 후 이를
Slave에 보내는 구조 설계

APB Master – 주요 신호

PRESETn – 0일 때 동작하는 리셋 신호

WREQ – CPU의 write 요청 신호

RREQ – CPU의 read 요청 신호

Rdata – slave에서 온 32비트 결과 CPU 전달

Ready – 다음 작업을 진행하도록 CPU 전달

PENABLE – APB 두 번째 사이클 알림

PSEL – 신호를 보낼 slave 선택(GPO, FND, UART ....)

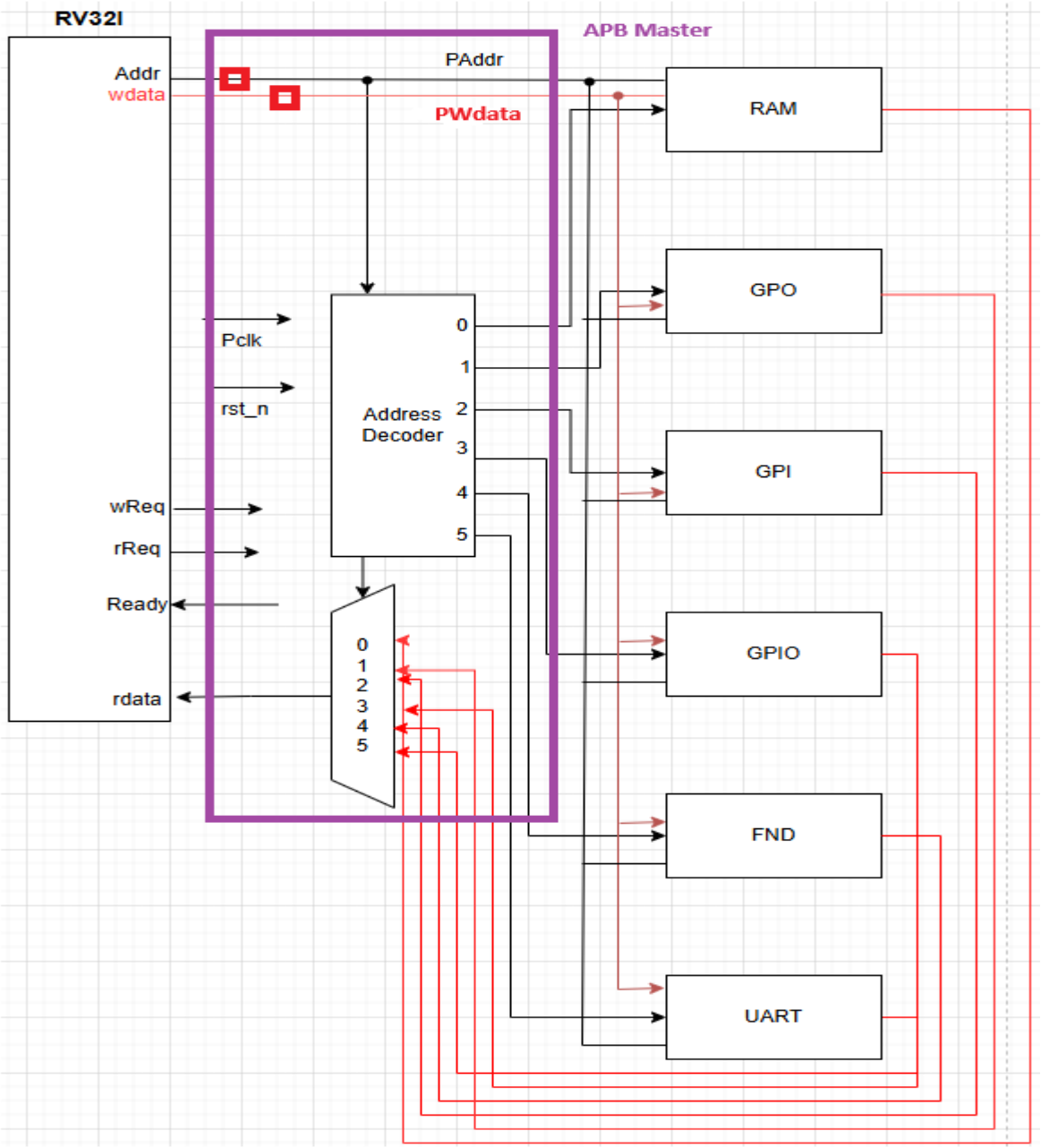
PREADY - 데이터 전송 완료 여부 master 전송

(spec document 참고)

```
module APB_Master (
    //BUS Global signal
    input  logic        PCLK,
    input  logic        PRESETn,
    //APB Master - CPU
    input  logic [31:0] Addr,
    input  logic [31:0] Wdata,
    input  logic        WREQ,    //from CPU(dwe)
    input  logic        RREQ,    //from CPU(dre)
    output logic [31:0] Rdata,
    output logic        Ready,
    //APB Master - Slave
    output logic [31:0] PADDR,
    output logic [31:0] PWDATA,
    output logic        PENABLE,
    output logic        PWRITE,
    output logic        PSEL,
    input  logic [31:0] PRDATA,
    input  logic        PREADY,
```

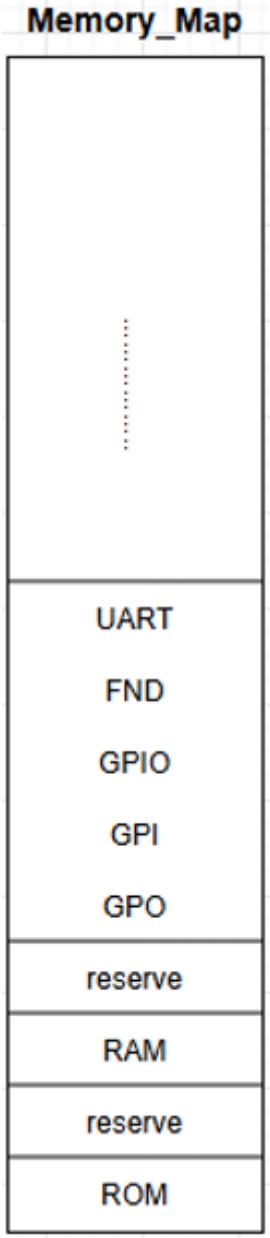
APB Master

각 Slave로 전달하는 decoder와
다시 받아서 RV32I로 보내는 MUX 구조 설계



APB Master

A31~A28	A15~A12	Hex 주소 범위	영역
0 0 0 0	0 0 0 0	0x0000_0000 ~ 0x0000_0FFF	1000h - ROM
0 0 0 1	0 0 0 0	0x1000_0000 ~ 0x1000_0FFF	1000h - RAM
0 0 1 0	0 0 0 0	0x2000_0000 시작	IO - GPO
0 0 1 0	0 0 0 1	0x2000_1000 ~	IO - GPI
0 0 1 0	0 0 1 0	0x2000_2000 ~	IO - GPIO
0 0 1 0	0 0 1 1	0x2000_3000 ~	IO - FND
0 0 1 0	0 1 0 0	0x2000_4FFF 종료	IO - UART



APB Master – Address Decoder

en – decoder의 동작을 제어

초기에 구성한 Memory map에 따라 각 Slave로 가는 조합 논리 설계

```
if (en) begin
    case (Addr[31:28])
        4'h1: begin
            if (Addr[27:12] == 16'h000_0) begin
                PSEL0 = 1'b1;  //RAM
            end
        end
        4'h2: begin
            case (Addr[15:12])
                4'h0: PSEL1 = 1'b1; //GPO
                4'h1: PSEL2 = 1'b1; //GPI
                4'h2: PSEL3 = 1'b1; //GPIO
                4'h3: PSEL4 = 1'b1; //FND
                4'h4: PSEL5 = 1'b1; //UART
            endcase
        end
    endcase
end
```

APB Master – MUX

Addr -> sel로 부터 받아

여러 개의 Slave 중 현재 주소에 해당하는 응답을 CPU로 전달

```
case (sel[31:28])
    4'h1: begin
        if (sel[27:12] == 16'h000_0) begin
            Rdata = PRDATA0;
            Ready = PREADY0;        //RAM
        end
    end
    4'h2: begin
        case (sel[15:12])
            4'h0: begin
                Rdata = PRDATA1;
                Ready = PREADY1;    //GPO
            end
            4'h1: begin
                Rdata = PRDATA2;
                Ready = PREADY2;    //GPI
            end
            4'h2: begin
                Rdata = PRDATA3;
                Ready = PREADY3;    //GPIO
            end
            4'h3: begin
                Rdata = PRDATA4;
                Ready = PREADY4;    //FND
            end
            4'h4: begin
                Rdata = PRDATA5;
                Ready = PREADY5;   //UART
            end
        endcase
    end
```

APB Master

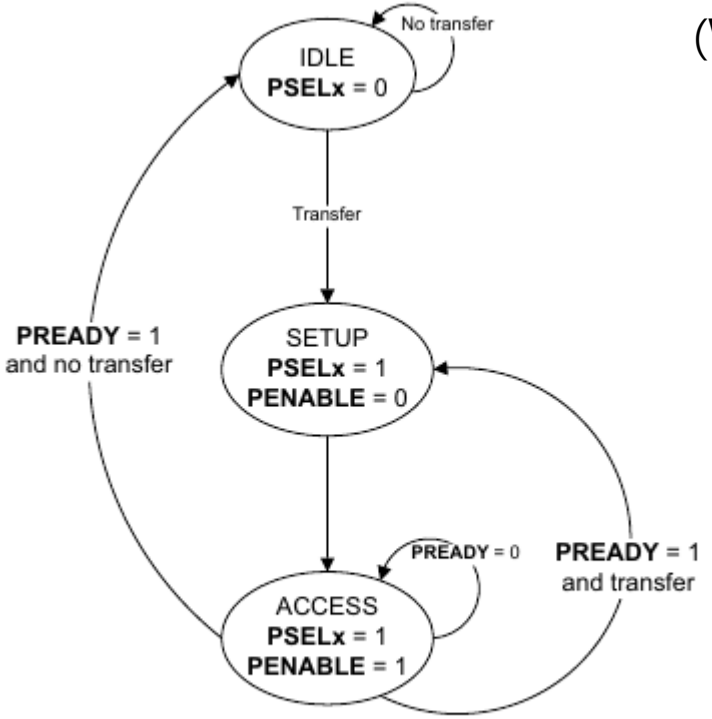


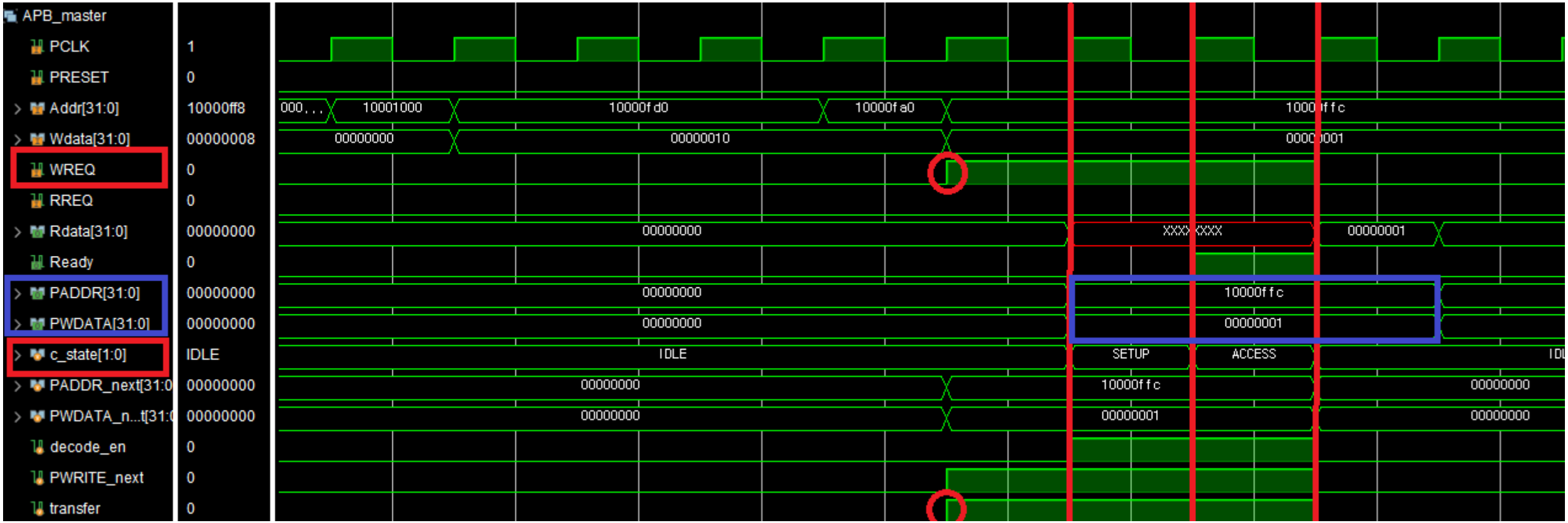
Figure 4-1 State diagram

from spec document

transfer – read or write
(WREQ | RREQ)

```
    case (c_state)
        IDLE: begin
            decode_en = 0;
            PENABLE = 0;
            PADDR_next = 32'd0;
            PWDATA_next = 32'd0;
            PWRITE_next = 1'b0;
            if (transfer) begin
                PADDR_next  = Addr;
                PWDATA_next = Wdata;
                if (WREQ) begin
                    PWRITE_next = 1'b1;
                end else begin
                    PWRITE_next = 1'b0;
                end
                n_state = SETUP;
            end
        end
        SETUP: begin
            decode_en = 1'b1;
            PENABLE   = 1'b0;
            n_state   = ACCESS;
        end
        ACCESS: begin
            decode_en = 1;
            PENABLE   = 1'b1;
            if (Ready) begin
                n_state = IDLE;
            end
        end
    endcase
end
```

APB Master Simulation



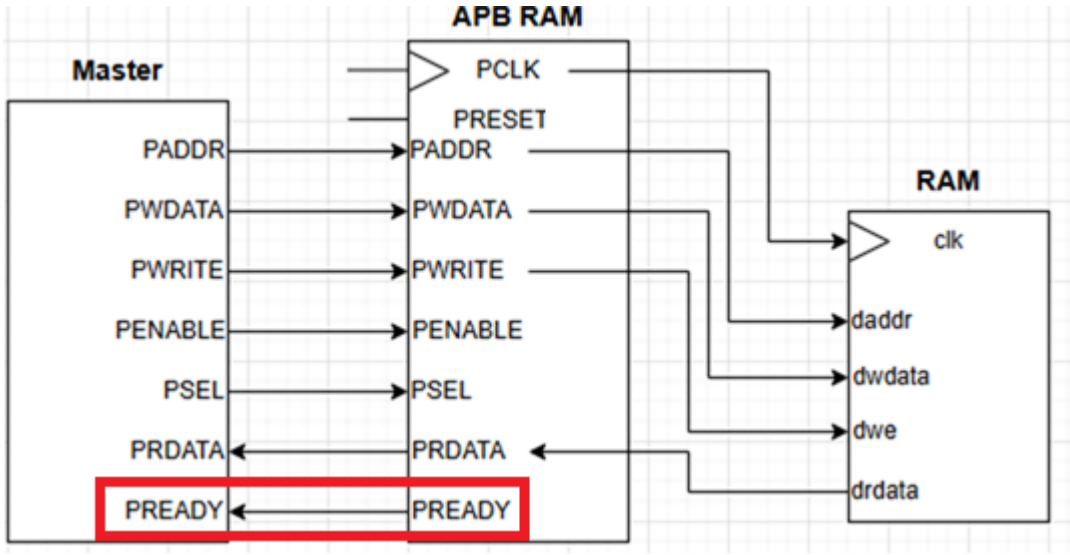
WREQ가 들어오면서 transfer high로 올라가고 즉시 IDLE -> SETUP 다음 클럭에 ACCESS
PADDR, PWDATA 주소를 SETUP ACCESS 까지 hold 시켜 데이터 샘플링

APB Slave - RAM

마스터가 보낸 주소(PADDR)과 데이터(PWDATA) 그대로 RAM으로 전달

PSEL, PENABLE, PWRITE 정보를 받아 dwe 활성화

마스터에서 신호가 들어오면 별도의 대기상태 없이 PREADY 신호 전송



APB Slave - RAM

32비트 World-aligned
Bmem 1024 -> 10비트만 필요

```
module APB_RAM (
    input                PCLK,
    input        [31:0] PADDR,
    input        [31:0] PWDATA,
    input               PENABLE,
    input               PWRITE,
    input               PSEL0,
    output logic [31:0] PRDATA0,
    output logic        PREADY0
);
    logic [31:0] bmem[0:1023];
    assign PREADY0 = (PENABLE & PSEL0) ? 1'b1 : 1'b0;

    always_ff @(posedge PCLK) begin
        if (PSEL0 & PENABLE & PWRITE) begin
            bmem[PADDR[11:2]] <= PWDATA;
        end
    end

    assign PRDATA0 = bmem[PADDR[11:2]];

endmodule
```

APB Slave – RAM Simulation

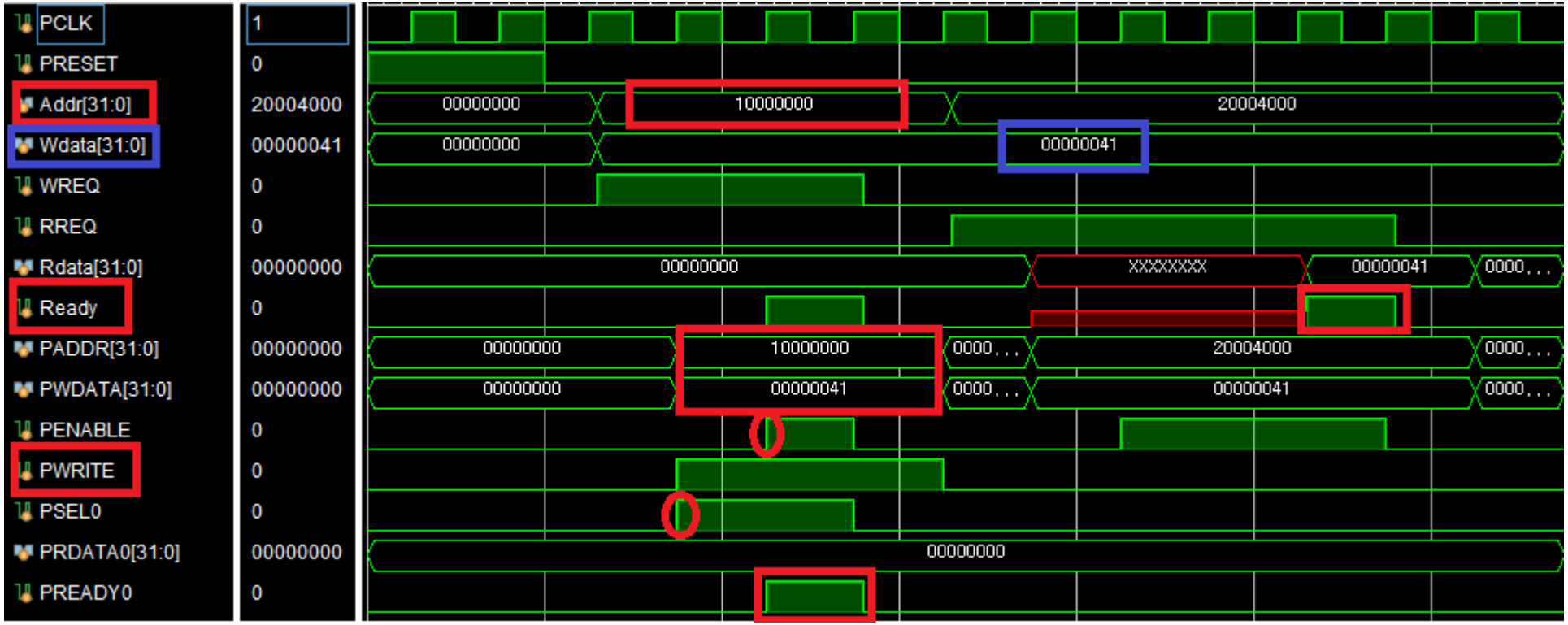
```
void main(void) {    Ram Base address
    *(unsigned int *) 0x10000000 = 0x00000041;
    return;
}
```

```
lui     a5,0x10000
li      a4,65
sw      a4,0(a5) # 10000000 <_sstack+0xffef600>
lw      a3,0(a5)
```

RAM 시작 주소에
0x41 데이터 넣는다
(Write)

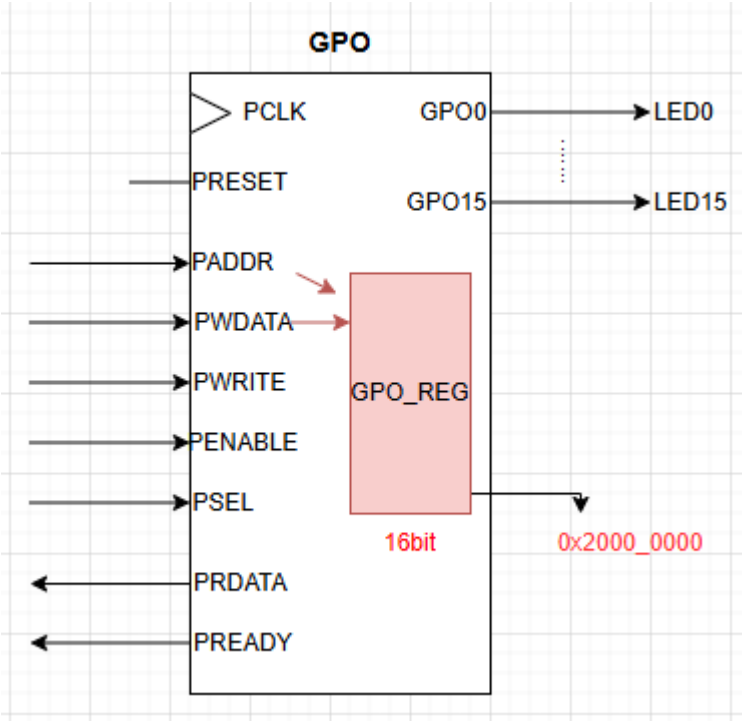
lui 0x1000을 밀어 넣고 하위 12비트 0
li 0x41 레지스터에 로드
sw 0x10000000에 0x00000041 쓰기 요청
lw 추가하여 read 모드 실행하여 cpu로 가져오기

APB Slave – RAM Simulation



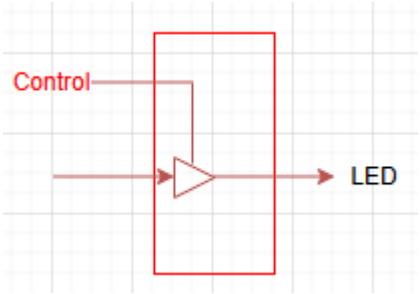
CPU의 요청(Addr, Wdata)가 실제 버스 신호(PADDR, PWDATA)로 전송되어 slave로 전달

APB Slave - GPO



GPO_CTL_REG – 0x2000_0000
(출력 활성화 레지스터)

GPO_ODATA_REG – 0x2000_0004
(실제 출력되는 16비트 데이터 레지스터)



Tri-state buffer

APB Slave - GPO

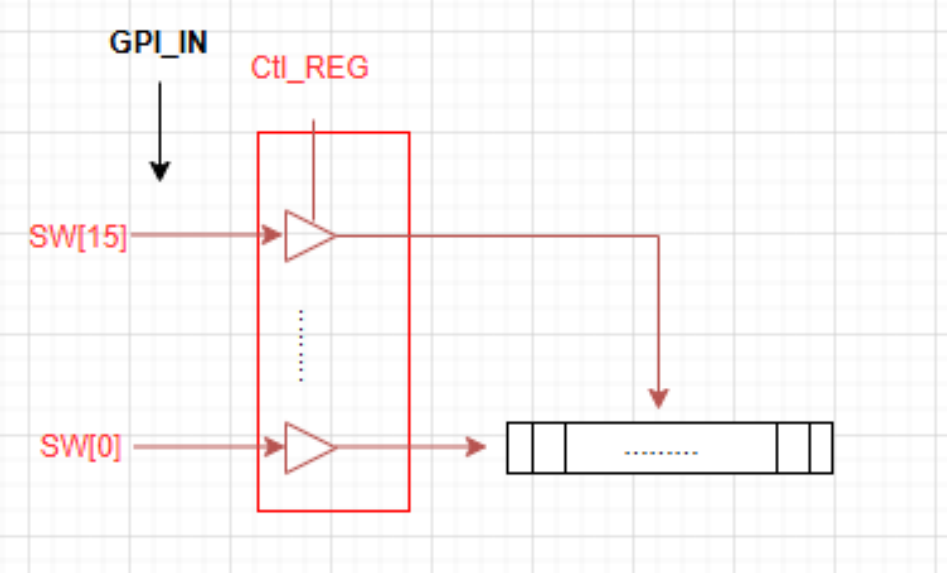
```
assign PREADY1 = (PSEL1 & PENABLE) ? 1'b1 : 1'b0;
assign PRDATA1 = (PADDR[11:0]== 12'h0000) ? {16'h0000,GPO_ctl_reg }:
        (PADDR[11:0]== 12'h0004) ? {16'h0000,GPO_odata_reg} :
        32'hxxxx_xxxx;
always_ff @(posedge PCLK or posedge PRESET) begin
    if (PRESET) begin
        GPO_ctl_reg   <= 16'd0;
        GPO_odata_reg <= 16'd0;
    end else begin
        if (PSEL1 & PENABLE & PWRITE) begin
            case (PADDR[11:0])
                12'h0000:   GPO_ctl_reg <= PWDATA[15:0];
                12'h0004: GPO_odata_reg <= PWDATA[15:0];
            endcase
        end
    end
end
```

```
genvar i;
generate
    for (i = 0; i < 16; i++) begin
        assign GPO_OUT[i] = (GPO_ctl_reg[i]) ? GPO_odata_reg[i] : 1'bz;
    end
endgenerate
```

출력 활성화 여부 레지스터와
출력되는 데이터 레지스터에 주소 저장

ctl_reg가 1이면 odata_reg 값이 그대로 나감
0이면 1'bz(high – impedance)

APB Slave - GPI



GPI_CTL_REG – 0x2000_1000
(입력 제어 레지스터)

GPI_IDATA_REG – 0x2000_1004
(실제 16비트 데이터 레지스터)

APB Slave - GPI

```
assign PREADY2 = (PSEL2 & PENABLE) ? 1'b1 : 1'b0;
assign PRDATA2 = (PADDR[11:0] == 12'h0000)   ? {16'h0000, GPI_ctl_reg} :
                 (PADDR[11:0] == 12'h0004) ? {16'h0000, GPI_idata_reg} :
                 32'hxxxx_xxxx;
always_ff @(posedge PCLK or posedge PRESET) begin
    if (PRESET) begin
        GPI_ctl_reg <= 16'd0;
    end else begin
        if (PSEL2 & PENABLE & PWRITE) begin
            if (PADDR[11:0] == 12'h0000) begin
                GPI_ctl_reg <= PWDATA[15:0];
            end
        end
    end
end
```

```
genvar i;
generate
    for (i = 0; i < 16; i++) begin
        assign GPI_idata_reg[i] = (GPI_ctl_reg[i]) ? GPI_IN[i] : 1'bz;
    end
endgenerate
```

입력 제어 여부 레지스터와
실제 보내는 데이터 레지스터에 주소 저장

GPI는 입력 전용 모듈이기에
항상 외부 핀 값을 읽어서 CPU에 전달

ctl_reg가 1이면 idata_reg 값이 그대로 나감
0이면 1'bz(high – impedance)

APB Slave - GPIO

inout 포트를 이용해 GPO와 GPI 하나의 모듈로 설계

```
assign PRDATA3 = (PADDR[11:0]== 12'h0000) ? {16'h0000,GPIO_ctl_reg }:
        (PADDR[11:0]== 12'h0004) ? {16'h0000,GPIO_odata_reg} :
        (PADDR[11:0]== 12'h0008) ? {16'h0000,GPIO_idata_reg} :32'hxxxx_xxxx;
always_ff @(posedge PCLK or posedge PRESET) begin
    if (PRESET) begin
        GPIO_ctl_reg   <= 16'd0;
        GPIO_odata_reg <= 16'd0;
    end else begin
        if (PSEL3 & PENABLE & PWRITE) begin
            case (PADDR[11:0])
                12'h0000:   GPIO_ctl_reg <= PWDATA[15:0];
                12'h0004: GPIO_odata_reg <= PWDATA[15:0];
            endcase
        end
    end
end
```

```
genvar i;
generate
    for (i = 0; i < 16; i++) begin
        assign GPIO[i]   = ctl[i] ? o_data[i] : 1'bz;
        assign i_data[i] = ~ctl[i] ? GPIO[i] : 1'bz;
    end
endgenerate
```

APB Slave – GPIO Simulation

설계한 모듈 전부 활용하기 위해

SW[0] ~ SW[7] : GPIO[0] ~ GPIO[7]
SW[8] ~ SW[15] : GPI_IN[0] ~ GPI_IN[7]

LED[0] ~ LED[7] : GPIO[8] ~ GPIO[15]
LED[8] ~ LED[15] : GPO_OUT[8] ~ GPO_OUT[15}

```
GPI_IN = 8'haa;
GPIO_SW = 8'h55;
repeat(100) @(negedge clk);
```

10101010_01010101

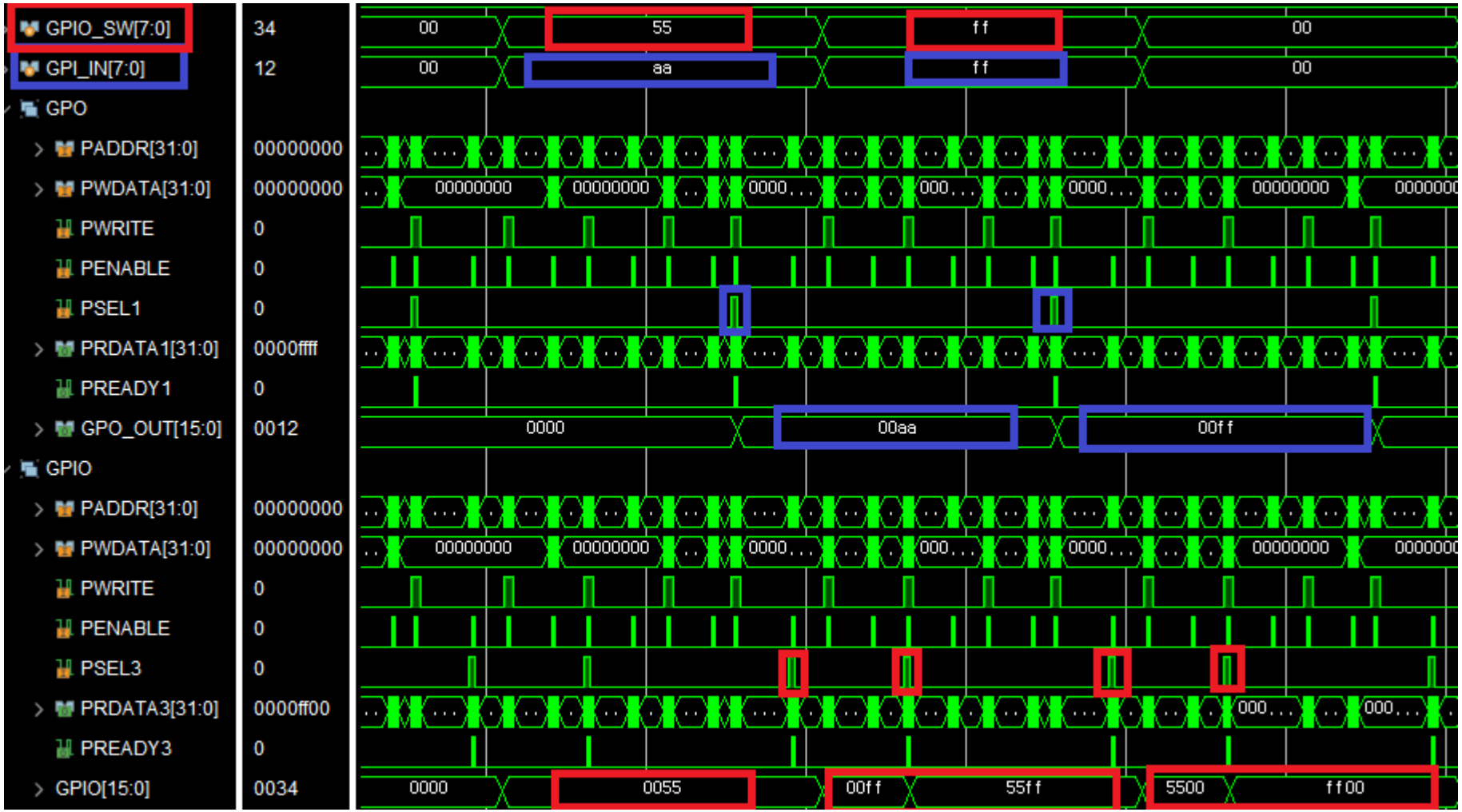
```
GPI_IN = 8'hff;
GPIO_SW = 8'hff;
repeat(100) @(negedge clk);
```

전부 on

```
GPI_IN = 8'h00;
GPIO_SW = 8'h00;
repeat(100) @(negedge clk);
```

전부 off

APB Slave – GPIO Simulation



GPIO 0~7 -> 8~15제어

GPI_IN -> GPO_OUT 제어

APB Slave – GPIO Simulation

C code

각 주소에 맞는 레지스터 지정

```c
#define GPO_CTL_REG     (*(volatile unsigned int *)0x20000000)
#define GPO_ODATA_REG   (*(volatile unsigned int *)0x20000004)
#define GPI_CTL_REG     (*(volatile unsigned int *)0x20001000)
#define GPI_IDATA_REG   (*(volatile unsigned int *)0x20001004)
#define GPIO_CTL_REG    (*(volatile unsigned int *)0x20002000)
#define GPIO_ODATA_REG  (*(volatile unsigned int *)0x20002004)
#define GPIO_IDATA_REG  (*(volatile unsigned int *)0x20002008)
```

16개의 GPIO 핀 중 상위 8비트(FF)는 출력으로 쓰고
하위 8비트(00)은 SW로

```c
int main() {
    unsigned int sw0_7, sw8_15;

    GPO_CTL_REG = 0xFFFF;
    GPI_CTL_REG = 0xFFFF;
    GPIO_CTL_REG = 0xFF00;

    while (1) {

        sw0_7 = GPIO_IDATA_REG & 0x00FF;
        GPIO_ODATA_REG = (sw0_7 << 8);

        sw8_15 = GPI_IDATA_REG & 0x00FF;
        GPO_ODATA_REG = sw8_15;
    }
    return 0;
}
```

APB Slave – GPIO Simulation

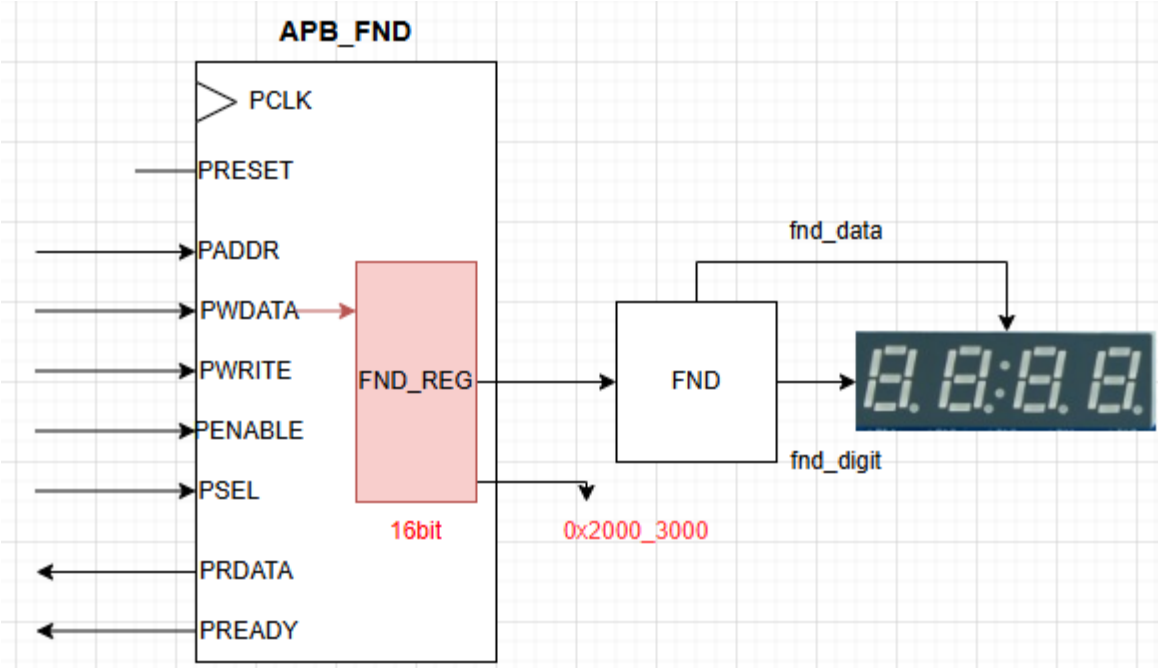
SW 전부 ON

내리면 LED OFF

APB Slave – F N D

CPU로부터 데이터를 받아
4자리의 7-Segment(FND)에 숫자 출력

4비트씩 총 4자리의 16진수 데이터 처리



APB Slave – F N D

```
module fnd_controller (
    input [15:0] sum,
    input clk,
    input reset,
    output [3:0] fnd_digit,
    output [7:0] fnd_data
);
```

숫자의 모양을 정하는 fnd_data
자릿수를 정하는 fnd_digit

```
always @(bcd) begin
    case (bcd)
        4'd0: fnd_data = 8'hC0;
        4'd1: fnd_data = 8'hf9;
        4'd2: fnd_data = 8'ha4;
        4'd3: fnd_data = 8'hB0;
        4'd4: fnd_data = 8'h99;
        4'd5: fnd_data = 8'h92;
        4'd6: fnd_data = 8'h82;
        4'd7: fnd_data = 8'hf8;
        4'd8: fnd_data = 8'h80;
        4'd9: fnd_data = 8'h90;
        4'hA: fnd_data = 8'h88; // A
        4'hB: fnd_data = 8'h83; // b
        4'hC: fnd_data = 8'hC6; // C
        4'hD: fnd_data = 8'hA1; // d
        4'hE: fnd_data = 8'h86; // E
        4'hF: fnd_data = 8'h8E; // F
        default: fnd_data = 8'hFF;
    endcase
end
```

16비트 표현하기 위해 모양 추가

APB Slave – F N D

```
assign PREADY4 = (PSEL4 & PENABLE) ? 1'b1 : 1'b0;
assign PRDATA4 = (PADDR[11:0] == 12'h0000) ? {16'h0000, fnd_reg}
:32'h0000_0000;

always_ff @(posedge PCLK or posedge PRESET) begin
    if (PRESET) begin
        fnd_reg <= 16'd0;
    end else begin
        if (PSEL4 & PENABLE & PWRITE) begin
            case (PADDR[11:0])
            12'h0000 : fnd_reg <= PWDATA[15:0];
            endcase
        end
    end
end
```

```
fnd_controller U_APB_FND(
    .sum(fnd_reg),
    .clk(PCLK),
    .reset(PRESET),
    .fnd_digit(fnd_digit),
    .fnd_data(fnd_data)
);
```

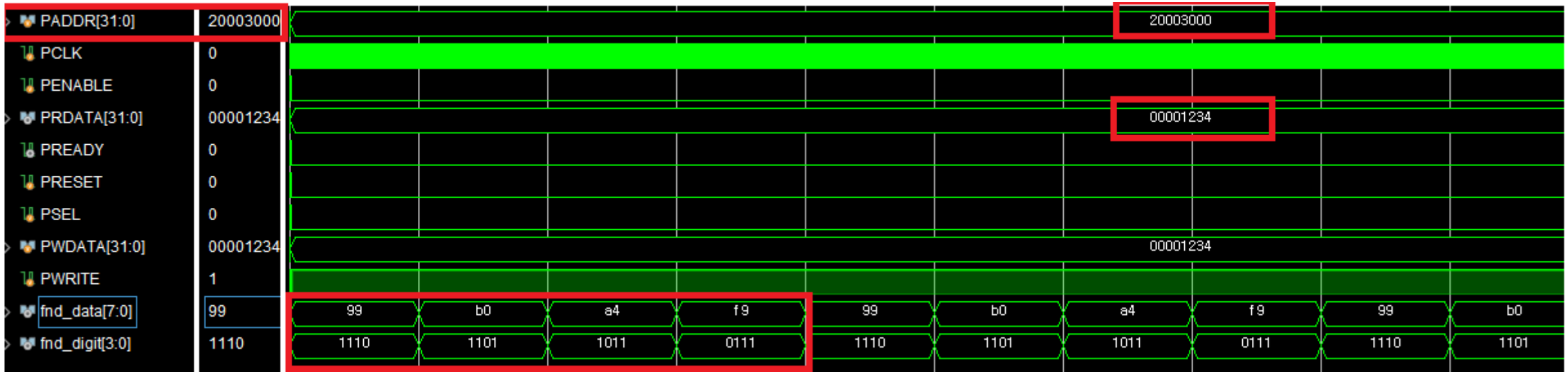
기존 Slave와 같은 방식으로 설계

인스턴스

APB Slave – F N D Simulation

FND로 '1234' 출력하기

FND memory map



(fnd_digit)	선택된 위치	(fnd_data)	나타내는 숫자
1110 (e)	1의 자리	99	4
1101 (d)	10의 자리	b0	3
1011 (b)	100의 자리	a4	2
0111 (7)	1000의 자리	f9	1

7-segment는 Acitve Low

APB Slave – Simulation

왼쪽 8개 스위치(GPI_IN) 오른쪽 8개(GPIO) 읽어옴

왼쪽 8개 LED(GPO_OUT) 오른쪽 8개(GPIO) 전송

합체된 16비트 데이터를 FND 레지스터로 전송

C code

```c
#include <stdint.h>

#define GPO_OUT_ADDR   0x20001000
#define GPI_IN_ADDR    0x20001004
#define GPIO_ADDR      0x20002000
#define FND_ADDR       0x20003004

#define GPO_OUT_REG    (*(volatile int *)GPO_OUT_ADDR)
#define GPI_IN_REG     (*(volatile int *)GPI_IN_ADDR)
#define GPIO_REG       (*(volatile int *)GPIO_ADDR)
#define FND_REG        (*(volatile int *)FND_ADDR)

int main(void) {
    int sw_left, sw_right, total_val;

    while (1) {

        sw_left = GPI_IN_REG & 0xFF;
        sw_right = GPIO_REG & 0xFF;


        GPO_OUT_REG = sw_left;
        GPIO_REG = (sw_right << 8);

        total_val = (sw_left << 8) | sw_right;
        FND_REG = total_val;
    }
    return 0;
}
```

APB Slave(GPIO, FND) – Simulation

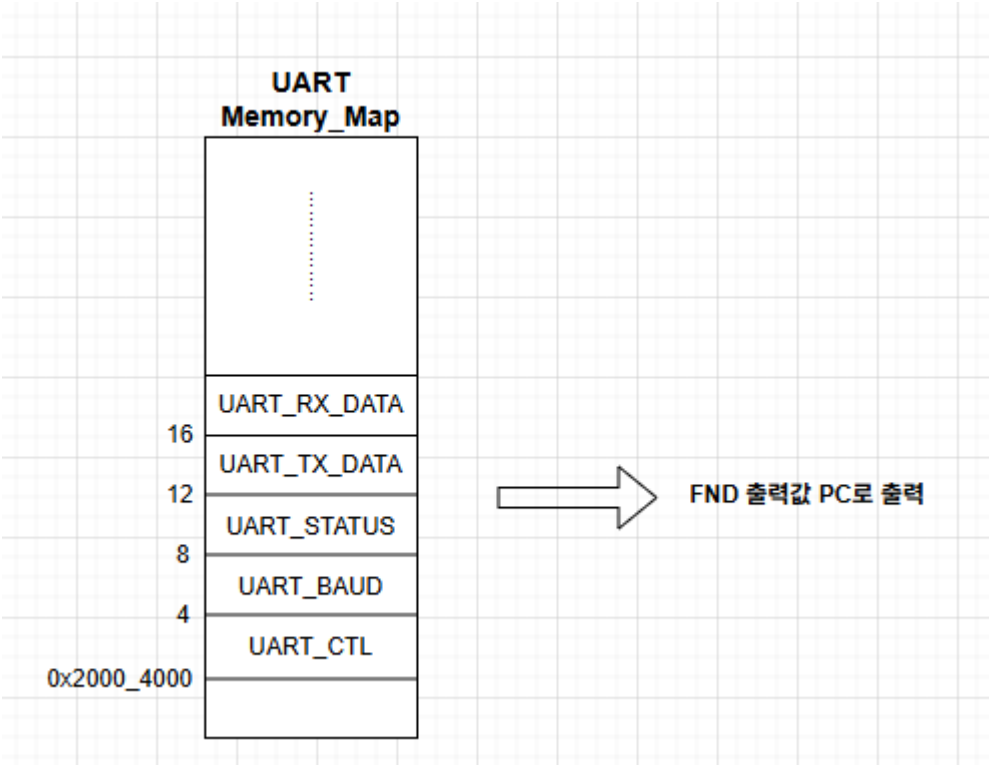
SW로 LED 제어 및 그에 맞는 FND 값 출력

APB Slave - UART

UART_CTL : 제어 레지스터

UART_BAUD : UART baud rate 결정(9600, 19200, 115200)

UART_STATUS : TX_READY와 RX_EMPTY 등 핵심 신호들을 전송하여 CPU에게 상태를 알림



APB Slave - UART

UART_STATUS

```
always_ff @(posedge PCLK or posedge PRESET) begin
    if (PRESET) begin
        uart_ctl_reg     <= 1'b0;
        uart_baud_reg    <= 2'b00;
        uart_tx_data_reg <= 8'h0;
        rx_done_hold     <= 1'b0;
    end else begin
        if (PSEL5 & PENABLE & PWRITE) begin
            case (PADDR[11:0])
                UART_CTL_ADDR:     uart_ctl_reg     <= PWDATA[0];
                UART_BAUD_ADDR:    uart_baud_reg    <= PWDATA[1:0];
                UART_TX_DATA_ADDR: uart_tx_data_reg <= PWDATA[7:0];
            endcase
        end
        else if (w_tx_busy) begin
            uart_ctl_reg <= 1'b0;
        end
        if (w_rx_done) begin
            rx_done_hold <= 1'b1;
        end
        else if  (PADDR[11:0] == UART_RX_DATA_ADDR) begin
            rx_done_hold <= 1'b0;
        end
    end
end
```

APB Slave - UART

```
// 100,000,000/(bps*16)
always_comb begin
    case (baud_sel)
        2'b00:   COUNT = 10'd651; // 9600 bps
        2'b01:   COUNT = 10'd325; // 19200 bps
        2'b10:   COUNT = 10'd54;  // 115200 bps
        default: COUNT = 10'd651; // 9600
    endcase
end
```

UART_BAUD에서 설정하기 위해
모듈 설계

```
b_tick BAUD_Gen (
    .clk(PCLK),
    .rst(PRESET),
    .baud_sel(uart_baud_reg),
    .b_tick(w_b_tick)
);
```

APB Slave - Simulation

C 코드 소프트웨어

```c
#include <stdint.h>

// [1] 메모리 맵 주소 정의
#define GPO_OUT_REG      (*(volatile uint32_t *)0x20001000)
#define GPI_IN_REG       (*(volatile uint32_t *)0x20001004)
#define GPIO_REG         (*(volatile uint32_t *)0x20002000)
#define FND_REG          (*(volatile uint32_t *)0x20003004)

// 추가된 UART 주소 (사진의 메모리 맵 기준)
#define UART_STATUS_REG  (*(volatile uint32_t *)0x20004008)
#define UART_TX_DATA_REG (*(volatile uint32_t *)0x2000400C)


// [2] UART 문자 송신 함수 (Polling 방식)
void uart_send_char(char data) {
    // UART_STATUS 레지스터의 0번 비트(TX_READY)가 1이 될 때까지 무한 대기
    while ((UART_STATUS_REG & 0x01) == 0) {
        // 대기 (하드웨어가 이전 문자 전송을 마칠 때까지)
    }
    UART_TX_DATA_REG = data; // 전송 가능해지면 데이터 쓰기
}
```

```c
// [2] UART 문자 송신 함수 (Polling 방식)
void uart_send_char(char data) {
    // UART_STATUS 레지스터의 0번 비트(TX_READY)가 1이 될 때까지 무한 대기
    while ((UART_STATUS_REG & 0x01) == 0) {
        // 대기 (하드웨어가 이전 문자 전송을 마칠 때까지)
    }
    UART_TX_DATA_REG = data; // 전송 가능해지면 데이터 쓰기
}

// [3] 16비트 숫자를 16진수 문자열로 예쁘게 보내는 함수
void uart_send_hex16(uint16_t val) {
    char hex_chars[] = "0123456789ABCDEF";

    uart_send_char('F');
    uart_send_char('N');
    uart_send_char('D');
    uart_send_char(':');
    uart_send_char(' ');

    // 4비트씩 잘라서 16진수 문자로 변환 (예: 0x1234 -> '1', '2', '3', '4')
    uart_send_char(hex_chars[(val >> 12) & 0xF]);
    uart_send_char(hex_chars[(val >> 8) & 0xF]);
    uart_send_char(hex_chars[(val >> 4) & 0xF]);
    uart_send_char(hex_chars[val & 0xF]);

    uart_send_char('\r'); // 캐리지 리턴 (줄의 맨 앞으로)
    uart_send_char('\n'); // 라인 피드 (다음 줄로)
}
```

APB Slave - Simulation

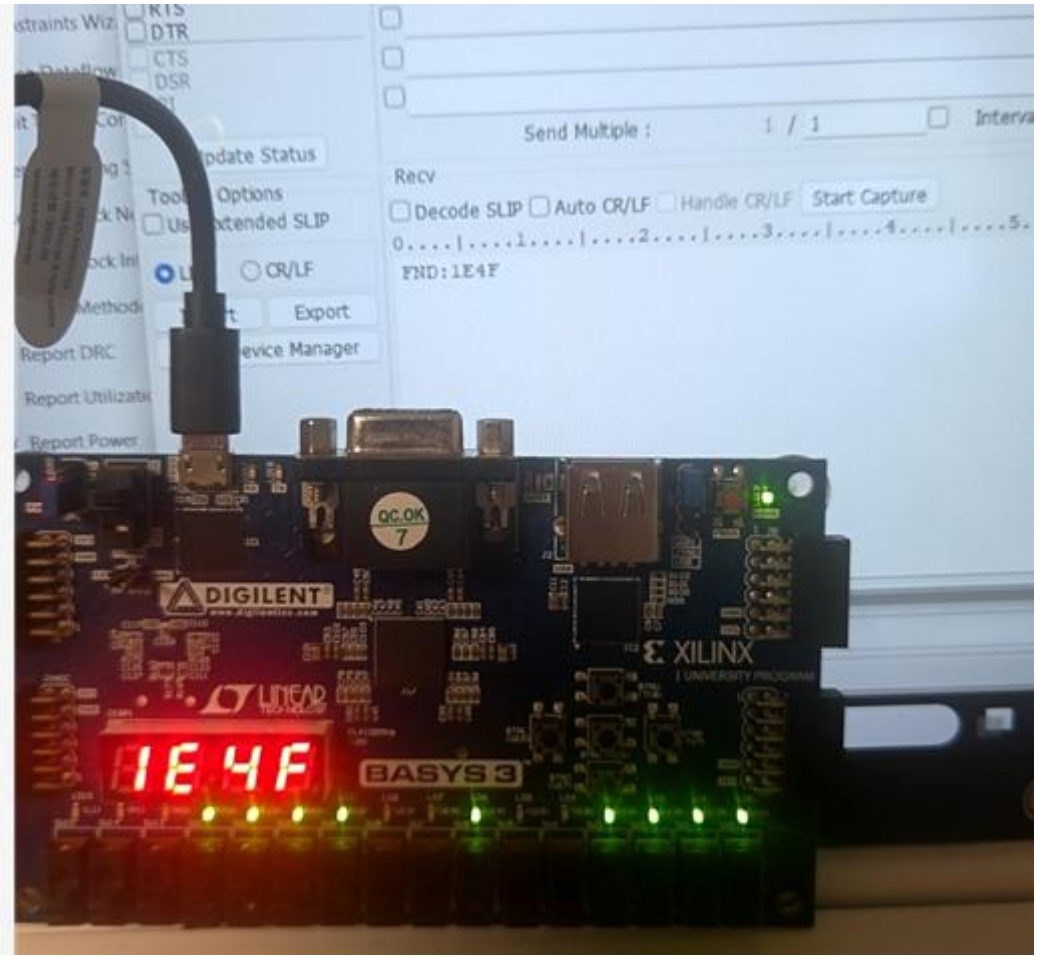
```c
int main(void) {
    uint32_t sw_left, sw_right, total_val;
    uint32_t prev_val = 0xFFFFFFFF; // 이전 값을 기억할 변수 (처음엔 무작위 세팅)

    while (1) {
        // 1. 스위치 값 읽기
        sw_left = GPI_IN_REG & 0xFF;
        sw_right = GPIO_REG & 0xFF;

        // 2. 16비트로 합치기
        total_val = (sw_left << 8) | sw_right;

        // 3. LED와 FND는 무조건 실시간 업데이트
        GPO_OUT_REG = sw_left;
        GPIO_REG = (sw_right << 8);
        FND_REG = total_val;

        // 4. UART 출력: 스위치 값이 '변했을 때만' PC로 전송
        if (total_val != prev_val) {
            uart_send_hex16(total_val); // PC 터미널로 "FND: 1234" 형태로 출력
            prev_val = total_val;       // 현재 값을 이전 값으로 저장
        }
    }
    return 0;
}
```



고찰 및 결론

1. 프로젝트 요약 - "RISC-V 기반 SoC 풀스택(HW/SW) 통합 설계 및 검증 완료"

- 하드웨어 IP 설계: AMBA APB 프로토콜의 규격(Setup/Access Phase)을 준수하여 RAM, GPIO, FND 컨트롤러, UART 다양한 주변장치(Peripheral) IP를 직접 RTL로 설계함.

- 통합 및 시뮬레이션 검증: RISC-V 코어 마스터와 직접 설계한 Slave 모듈들을 연결하고 파형 분석을 통해 버스 트랜잭션 및 구동타이밍 안정성을 검증함.

- 소프트웨어 제어 및 보드 검증: Memory-Mapped I/O 방식을 활용하여 하드웨어 레지스터를 제어하는 C 코드 작성하고 FPGA 보드 상에서 스위치, LED, FND, PC 터미널 간의 실시간 상호작용을 성공적으로 구현함.

2. 기술적 고찰

- 하드웨어-소프트웨어 공동 설계(HW/SW Co-design)의 중요성 인식: 하드웨어가 제공하는 제어/상태/데이터 레지스터의 메모리 맵이 소프트웨어 구조를 결정한다는 것을 깨달음.

- 초고속으로 동작하는 CPU와 상대적으로 느린 주변장치(FND 디스플레이, UART 통신) 간의 속도 차이를 해결하기 위해, 하드웨어의 상태 플래그(UART_STATUS)를 소프트웨어에서 폴링(Polling) 방식으로 지속 확인하며 동기화를 맞추는 기법을 습득함.

3. 한계점 및 향후 발전 방향

- 현재 구조는 CPU가 무한 루프 안에서 스위치와 UART 상태를 계속 감시하는 폴링 방식을 사용하여 CPU 리소스 점유율이 높음. 향후 RISC-V의 인터럽트(IRQ) 컨트롤러를 구현하여 이벤트가 발생했을 때만 CPU가 동작하도록 시스템 효율을 높이는 설계 필요

- 추후에 더 높은 대역폭이 필요한 장치를 위해 APB 구조를 넘어 AMBA AHB 또는 AXI 버스 프로토콜로의 확장을 하고자 함

참고 문헌

AMBA APB Protocol Specification.pdf
- **Copyright 2003-2023 Arm Ltd. All rights reserved ARM IHI 0023E(ID022823)**

AMBA Specification (Rev 2.0)
- **Copyright 1999 ARM Limited. All rights reserved ARM IHI 0011A**